Slide 1: UIC School Clinic Management System

CONTENT

Verified presentation source | October 26, 2026

- A Java desktop application for school-clinic operations.
- Staff workflows for patient records, consultations, inventory, dispensing, notifications, and reports.
- Prepared from the current project source and implementation; use synthetic or approved demo data during presentation.

Implementation references: README.md; src/clinic/app/GUI.java; src/clinic/app/Router.java

Slide 2: Purpose and Project Scope

CONTENT

- The project brings core clinic record and medicine workflows into one role-aware desktop application.
- Scope includes staff sign-in, student and employee registration, visit documentation, stock tracking, dispensing, alerts, notifications, and administration.
- Patients and guardians do not have self-service application accounts.
- No measured paper-process baseline or before-and-after outcome data was supplied; no time-saving or error-reduction claim is made.

Implementation references: README.md; src/clinic/app/Router.java; src/clinic/app/nav/*

Slide 3: Implemented System Objectives

CONTENT

- Authenticate staff and apply role permissions to clinic and administrative functions.
- Maintain student and employee profiles, individually stored allergies, and a separate optional guardian-contact field.
- Record consultations with controlled multi-symptom selection, a required concern, and optional remarks.
- Track medicine batches, expiry, stock movements, refill history, and dispensing transactions.
- Support operational review through reports, stock/visit alerts, audit entries, and an optional nurse-only AI assistant.

Implementation references: src/clinic/core/ClinicSystem.java; src/clinic/service/ClinicData.java; src/clinic/app/ui/*

Slide 4: Users and Access Roles

CONTENT

- Nurse: patient and consultation workflows, dispensing, stock view and restock, reports, dashboard, and AI assistant.
- Nurse assistant: patient and consultation workflows, dispensing, and stock view; restocking and reports are not granted.
- Doctor: report-viewing workspace; no patient-registration or dispensing permission.
- Administrator (shown as Superadmin in navigation): user and campus management plus audit-log access; no clinical workflow access.
- Navigation visibility is paired with service-level authorization; role checks are not only visual.

Implementation references: src/clinic/model/{User,SchoolNurse,NurseAssistant,Doctor,Administrator}.java; src/clinic/app/nav/*; src/clinic/service/SecuredClinicSystem.java

Slide 5: Main Functional Modules

CONTENT

- Patients: register Student or Employee records, search profiles, view details, and add allergies.
- Consultations: record registered-patient visits with one or more controlled symptom labels, concern, remarks, time, and staff ID.
- Dispensing and inventory: manage medicine batches, check safety and usable stock, record transactions, and view history.
- Notifications and reports: prepare transaction-linked clinic messages for manual delivery; view and format stored visit reports.
- Administration and dashboard: manage staff accounts/campuses, review audit events, and see clinic summaries.

Implementation references: src/clinic/app/ui/patients/PatientsPanel.java; consultations/; dispense/; inventory/; notifications/; reports/; ui/admin/

Slide 6: Typical Clinic Workflow

CONTENT

- 1. An administrator completes first-run setup and creates staff accounts; staff sign in with their assigned role.
- 2. Staff register a Student or Employee using the patient ID and profile fields; allergies are entered as separate items.
- 3. Staff document a consultation by choosing symptoms from the controlled list and entering the concern.
- 4. For dispensing, staff select a patient and medicine items; the application checks allergy records, medicine availability, expiry, and requested quantity.
- 5. Staff review visit and transaction history, prepare a guardian/employee notification if needed, then mark it sent only after external delivery.

Implementation references: src/clinic/app/ui/auth/LoginPanel.java; patients/; consultations/; dispense/; notifications/; src/clinic/service/DispenseWorkflow.java

Slide 7: Inventory and Dispensing Controls

CONTENT

- Medicine is stored as batches with stable UUID identity, quantity, expiry date, batch date, and remarks.
- Matching unexpired batches can be merged or kept separate after confirmation; staff can also review existing duplicate groups.
- A zero-stock batch can be refilled in place after confirmation; the batch keeps its ID and receives the new expiry, remarks, quantity, and refill date.
- Dispensing preflights requested items, consumes usable stock by earliest expiry, and reports completed versus unattempted items if a later row fails.
- Allergy confirmation does not bypass expired-stock or out-of-stock checks.

Implementation references: src/clinic/model/Medicine.java; src/clinic/service/Inventory.java; DispenseWorkflow.java; src/clinic/app/ui/inventory/InventoryPanel.java

Slide 8: Alerts, Forecasts, and Reports

CONTENT

- Low-stock alert: at most 10 usable units in aggregate across a medicine name.
- Expiry alert: usable batches expiring within 30 days; expired batches with stock are separately surfaced.
- Visit-pattern rules: 3 same-symptom visits in a student section or employee department within 48 hours; recurring single-patient patterns use 3 visits in 14 days.
- Medicine-days estimate uses recorded dispensing over a 30-day window; with no recorded use there is no finite rate-based estimate.
- Reports list visit records and can format a selected record; a full export-to-PDF/Excel workflow is not implemented.

Implementation references: src/clinic/service/FeatureRules.java; PatternDetector.java; RestockForecast.java; AlertCollector.java; src/clinic/app/ui/reports/ReportsPanel.java

Slide 9: Core Data Managed by the System

CONTENT

- Patient: ID, name, own contact, Student/Employee type, course or department, section or position, and student year level.
- Guardian contact is stored separately from the patient contact; allergies are saved as individual values.
- Consultation: record ID, patient ID, symptom list, concern, remarks/assessment, staff ID, and visit timestamp.
- Medicine transaction: transaction ID, medicine, patient and staff IDs, quantity, expiry, remarks, and timestamp.
- Notifications reference a patient and dispensing transaction; an employee notice uses the employee contact, while a student notice requires guardian contact.

Implementation references: src/clinic/model/{Patient,Student,Employee,MedicalRecord,MedicineTransaction,NotificationLog}.java; src/clinic/service/NotificationStore.java

Slide 10: Application Architecture

CONTENT

- Presentation: Java Swing panels, shared forms/dialogs, paged tables, role-aware navigation, and background UI work.
- Composition and session: ApplicationContext assembles login, audit, admin services, clinic data, and change notifications.
- Domain/service: ClinicSystem is the clinic operation contract; secured services enforce permissions and domain services implement validation and workflow rules.
- Data access: ClinicData adds internal query/batch operations; the in-memory implementation supports fixtures, while ClinicRepository adapts operations to JDBC persistence.
- Data-change topics refresh relevant tables and alerts after successful operations.

Slide 11: Technology, Persistence, and Security

CONTENT

- Java 21 with plain Java Swing/AWT; the project uses javac and Windows CMD/PowerShell launchers, not Maven or Gradle.
- Embedded H2 2.5.252 JDBC database; normal startup stores records locally under data/scms unless clinic.database overrides the path.
- Database file encryption uses H2 AES mode with a separately stored local key file; keep the database and matching key together.
- Passwords use salted PBKDF2WithHmacSHA256; sign-in tracks failed attempts and temporary lockout. Role checks and audit logging are implemented in services.
- Demo and automated fixtures can use in-memory or isolated temporary data; they are distinct from normal persistent startup.

Implementation references: README.md; src/clinic/core/{Login,PasswordHasher}.java; src/clinic/persistence/{Database,Schema}.java; tools/dependencies.cmd

Slide 12: Optional AI Nurse Assistant

CONTENT

- The nurse-only panel can send the current typed question to Google Gemini over HTTPS when GEMINI_API_KEY is configured; GEMINI_MODEL can select a provider-supported model.
- The assistant receives no SCMS database context, cannot execute clinic actions, and is instructed not to diagnose or prescribe.
- The UI requires a non-identifying-data confirmation; the client also screens for some obvious email, phone, and labeled-ID patterns.
- Pattern screening is not reliable de-identification. Do not submit patient names, IDs, contact details, records, or credentials.
- Live provider success and institutional approval for sending data to an external service are not verified by this project documentation.

Implementation references: README.md; src/clinic/ai/GeminiAiService.java; src/clinic/app/ui/assistant/AiNurseAssistantPanel.java

Slide 13: Suggested Live Demonstration

CONTENT

- Use a demo or approved synthetic dataset; sign in and show how the workspace changes by role.
- Register a synthetic Student with separately entered allergies, then record a consultation with multiple selected symptoms.
- Show inventory quantity/expiry, duplicate-batch choice, and refill of a zero-stock batch.
- Run a multi-item dispense: review the allergy confirmation and stock/expiry behavior, then inspect transaction history.
- Show a visit report and a locally generated notification; confirm delivery only after demonstrating the manual external-delivery step.

Slide 14: Limitations and Responsible Use

CONTENT

- This is a local desktop project, not a certified clinical decision-support or production health-record service.
- Safety checking flags recorded allergies for staff review; it does not compare medicine ingredients with allergy names or provide clinical advice.
- Guardian messages are generated locally; email/SMS delivery is not integrated and staff must deliver them outside the application.
- Appointments, patient deletion/general profile editing, password recovery, and a managed multi-user server deployment are not present in the current workflow.
- Accessibility, institution-wide privacy approval, backup/restore operations, and live AI-provider reliability require separate evaluation.

Implementation references: src/clinic/service/SafetyChecker.java; src/clinic/service/NotificationStore.java; docs/SCMS_AUDIT_2026-10-25.md (cross-checked against current code)

Slide 15: Conclusion

CONTENT

- SCMS connects staff sign-in, patient and visit records, inventory, dispensing, notifications, reports, and administration in one Java desktop application.
- Its implemented strengths are role-aware service checks, batch/expiry handling, local persistence, audit events, and reusable Swing workflows.
- Present the system with synthetic or institutionally approved data and describe it as an academic/local prototype.
- Next steps should be set by approved project requirements, privacy review, operational recovery needs, accessibility evaluation, and user feedback.

Implementation references: Current project implementation; README.md; docs/SCMS_AUDIT_2026-10-25.md